Ticket-Watcher

Thema: Geschäftsprozesse mit Camunda-Plattform

Dokumentinformationen

Dateiname: documentation.docx

Speicherdatum: 31.03.2026

Autoren: Luka Grujic, Elif Berra Canmaya

Inhaltsverzeichnis

Referenzdokumente 2
1 Einleitung 3
1.1 Auftragsbeschreibung 3
1.2 Setup 3
2 Vorgehensweise 6
3 Umsetzung 7
3.1 Architektur 7
3.2 Error-Handling 7
3.3 Tests 7
3.4 Technical Debt 7
4 Fazit 9
4.1 Schwierigkeiten 9
4.2 Reflektion 9

Referenzdokumente

[1] backend-structure.drawio

[2] todo-list_concert-ticket-watcher.txt

1 Einleitung

1.1 Auftragsbeschreibung

Im Rahmen dieses betrieblichen Auftrags wurde eine automatisierte Engine entwickelt, die Nutzer benachrichtigt, sobald Konzerttickets für einen gewünschten Künstler in ihrer Nähe verfügbar sind. Die Idee entstand aus dem alltäglichen Problem, dass Konzerte oft ausverkauft sind, bevor man von deren Existenz erfährt. Die Engine übernimmt das kontinuierliche Überwachen und Prüfen vollständig automatisiert.

1.1.1 Zentral Anforderungen

Nr.	Anforderungen	Umsetzung durch
1	Standort des Nutzers automatisch ermitteln	GeolocationFetcher (ip-api.com)
2	Konzerte eines Künstlers in der Nähe suchen	TicketmasterEventFetcher
3	Eingeladene Nutzer validieren	InviteeTaskService
4	Ticket-Verfügbarkeit eines Events prüfen	TicketmasterTicketFetcher
5	Nutzerdaten persistent speichern	UserRepository (PostgreSQL)
6	Fehlgeschlagene Aufgaben automatisch wiederholen	Camunda handleFailure()

1.2 Setup

1

PostgreSQL Container starten

```
docker run --name concert-db -e POSTGRES_DB=concertdb -e
POSTGRES_USER=admin -e POSTGRES_PASSWORD=admin -p 5432:5432 -d
postgres
```

2

Camunda Container starten

```
docker run --name camunda -p 8080:8080 -d camunda/camunda-bpm
platform:7.20.0
```

3

Camunda öffnen und einloggen

http://localhost:8080/camunda (Username: demo | Password: demo)

4

BPMN deployen

Camunda Modeler öffnen, ConcertTicketWatcher.bpmn laden und zusammen mit allen. form-Dateien auf *http://localhost:8080/engine-rest* deployen

5

Benutzer anlegen

Gehe zu *http://localhost:8080/camunda/app/admin*. Mindestens zwei Benutzer mit E-Mail-Adresse erstellen.

Beim Einladen wird die Benutzer-ID verwendet, nicht der Anzeigename. z.B. john, nicht John Doe. Alle eingeladenen Benutzer benötigen Zugriffsrechte auf die Tasklist-Applikation sowie auf die Prozessdefinition. Diese können unter Admin, dann Authorizations konfiguriert werden.

6

Java-Backend-Projekt öffnen

Mit IntelliJ öffnen -> Gradle-Dependencies neu laden.

7

Datenbank in IntelliJ verbinden

In IntelliJ rechts das Database-Panel öffnen und eine neue PostgreSQL-Datenquelle hinzufügen.

IntelliJ wird beim ersten Mal den passenden JDBC-Treiber automatisch zum Download anbieten. Diesen dann auch herunterladen.

Verbindungseinstellungen:
Host localhost: Port 5432
Datenbank: concertdb
Benutzer: admin
Passwort: admin
(Die Verbindung ist optional, hilft)

8

Config File ausfüllen

In src/main/resources/ die Datei config.example.properties kopieren und in config.properties umbenennen. Anschliessend folgende Werte eintragen:

1. Ticketmaster API-Key (erhältlich unter *https://developer.ticketmaster.com*)

2. Gmail-Adresse sowie ein Gmail App-Passwort. Das App-Passwort wird in den Google-Kontoeinstellungen unter Sicherheit -> 2-Faktor-Authentifizierung erstellt. Ein normales Passwort funktioniert hier nicht.

9

Java-Projekt starten

Main.java ausführen. Wenn alles korrekt konfiguriert ist, erscheinen im Log nacheinander

```
[SUCCESS] -> Database schema initialized successfully.
[SUCCESS] -> DB setup done
[SUCCESS] -> Components setup done
[SUCCESS] -> Camunda setup done
[SUCCESS] -> All setup done
```

Danach erscheinen laufend INFO- und DEBUG-Meldungen. Das ist normal und zeigt, dass das Backend auf Camunda-Tasks wartet. Das Backend muss während des gesamten Prozesses laufen. Wird es gestoppt, bleiben laufende Prozessinstanzen in Camunda hängen.

Ebenfalls wichtig zu wissen: Das Backend brauch bis zu 30s, um Tasks zu erkennen

10

Prozess starten

http://localhost:8080/camunda/app/tasklist öffnen und den Prozess starten.

11

Fertig

Von hier an sollte der Concert Ticket Watcher laufen und selbsterklärend sein.

2 Vorgehensweise

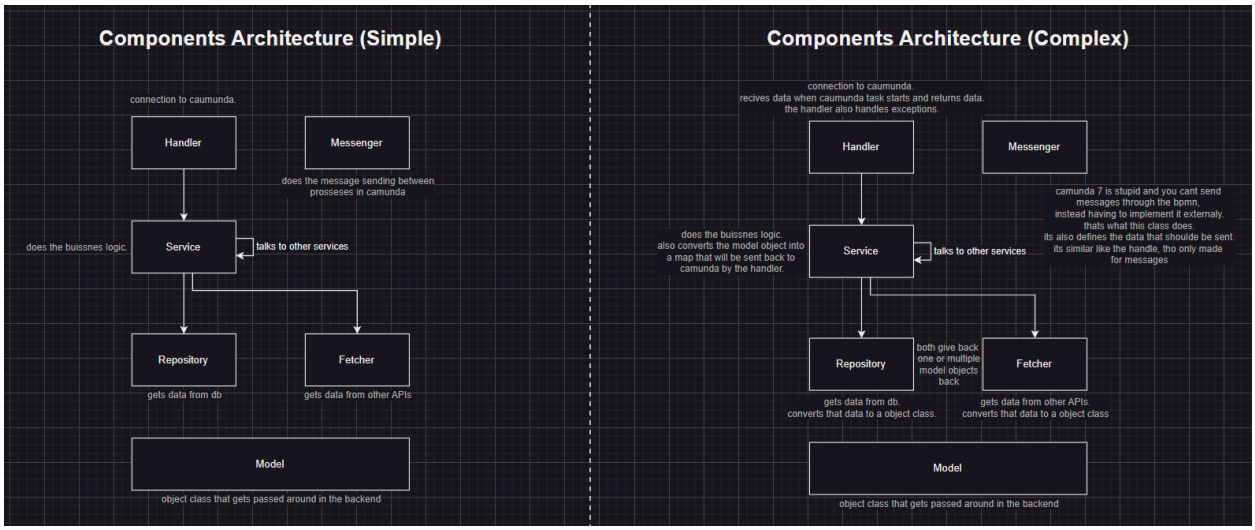
Das Projekt wurde in mehreren Phasen durchgeführt. In der ersten Phase wurde die Architektur des Prozesses mit dem Camunda Modeler definiert und iterativ verfeinert. Parallel dazu wurden die Anforderungen an das Java-Backend festgelegt. In der zweiten Phase erfolgte die schrittweise Implementierung, beginnend mit den abstrakten Basisklassen der generischen Schicht, gefolgt von den konkreten Messengers, Fetchers, Repositories, Services und Handlers. In der dritten Phase wurde das Backend mit dem Camunda-Prozess verbunden und ausgiebig end-to-end getestet. Als gemeinsames Planungswerkzeug haben wir eine txt Todo-Liste (was im Anhang ist) erstellt und auf gute Kommunikation vertraut.

3 Umsetzung

3.1 Architektur

Das Backend ist nach einem Spring-Boot-inspirierten Schichtmodell aufgebaut, ohne Spring Boot selbst zu verwenden. Es besteht aus vier Schichten: Handlers empfangen Aufgaben von Camunda und delegieren sie an Services. Services enthalten die eigentliche Geschäftslogik und rufen Fetchers auf. Fetchers sind für externe API-Aufrufe zuständig und geben typisierte Modelle zurück. Repositories verwalten den Datenbankzugriff. Die gesamte Verdrahtung der Abhängigkeiten erfolgt manuell in der Setup-Klasse. Eine Übersicht der Architektur ist im Referenzdokument backend-architektur.drawio zu finden.

Zudem sollten sowohl das BPMN-Diagramm als auch das Backend ausreichend mit Kommentaren verseht sein, die weit ins Detail spezifische Elemente erklären.



3.2 Error-Handling

Das System unterscheidet zwischen fatalen Fehlern und wiederkehrenden Fehlern. Fatale Fehler, wie ein nicht erreichbarer Geolocation-Dienst oder ein Datenbankfehler, führen zu einem sofortigen Abbruch des Prozesses. Wiederkehrende Fehler, wie ein nicht gefundenes Konzert oder noch nicht verfügbare Tickets, werden stillschweigend protokolliert und beim nächsten Zyklus erneut versucht. Der Nutzer wird bei einem Absturz des Prozesses nicht direkt informiert, da kein eigenes Benachrichtigungssystem für Fehler implementiert wurde in Camunda. Jedoch das Backend informiert über jeden Error.

3.3 Tests

Auf formelle Unit-Tests wurde verzichtet. Das System wurde stattdessen durch wiederholtes manuelles Durchspielen des gesamten Prozesses getestet, inklusive gezielter Edge Cases wie ungültige Benutzernamen, nicht existierende Künstler, mehrere gleichzeitig laufende Prozessinstanzen und das Einladungssystem mit mehreren Invitees. Nicht alle denkbaren Edge Cases sind abgedeckt, insbesondere Szenarien mit Netzwerkausfällen während laufender Prozesse oder gleichzeitigen Schreibzugriffen auf die Datenbank wurden nicht systematisch geprüft.

3.4 Technical Debt

Da es sich um ein Schulprojekt mit begrenztem Zeitrahmen handelt, wurden Technical Debts bewusst in Kauf genommen.

3.4.1 Silent Failure

Ungültige oder nicht registrierte Einladungs-Benutzernamen werden stillschweigend ignoriert, ohne den Benutzer zu informieren. Der Concert Ticket Watcher wird nicht crashen, dies ist weiterhing robust.

3.4.2 Alles nur auf einem Computer

Das gesamte System läuft auf einer einzigen lokalen Maschine. Kommunikation zwischen mehreren Computern wird nicht unterstützt.

3.4.3 Nicht das effizienteste Backend

Kein Global-Class-Handler: Alle Klassen müssen in Setup.java manuell instanziiert und injiziert werden. Es gibt kein automatisches Dependency-Injection-System wie in Spring Boot, das Klassen selbstständig erkennt und verdrahtet.

Kein DTO-System: Input und Output werden als einfache String-Schlüssellisten definiert. Es gibt keine Garantie zur Kompilierzeit, dass die Schlüssel mit den BPMN-Prozessvariablen übereinstimmen.

Schlechtes Error-Handling: Fast alle Fehler werden als generische RuntimeException weitergegeben. Es wird ebenfalls nicht zwischen behebbaren und nicht behebbaren Fehlern unterschieden.

Keine Unit-Tests wurden geschrieben.

Aufgrund des ungeplanten Gebrauches von Messengers, sind diese nicht so schön aussehend wie der Rest des Systems.

4 Fazit

4.1 Schwierigkeiten

Eine der grössten Hürden war das Deployment des BPMN-Prozesses in Camunda 7.20. Verschiedene Formular-Komponenten wie Text-Elemente, Separator-Felder und Bild-Elemente, die im Camunda Modeler ohne Fehlermeldung erstellt werden können, werden von der Camunda-7.20-Engine zur Laufzeit nicht unterstützt. Dies führte zu wiederholten Deployment-Fehlern, die nur durch manuelles Bereinigen der Formulardateien behoben werden konnten.

Eine weitere grosse Herausforderung war die Kommunikation zwischen mehreren Camunda-Prozess-Pools. In Camunda 7 werden Message Flows zwischen Pools zur Laufzeit nicht automatisch ausgeführt. Sie müssen explizit über die REST-API ausgelöst werden. Dies wurde durch eine eigene Messenger-Schicht im Backend gelöst, die intermediate Throw Events als externe Tasks verarbeitet und die entsprechenden Nachrichten gezielt korreliert. Besonders komplex war die Korrelation bei Multi-Instance Sub-Prozessen mit mehreren gleichzeitig wartenden Executions, da Camunda keine direkte Möglichkeit bietet, eine spezifische Execution innerhalb desselben Prozesses über den Nachrichtennamen allein anzusprechen.

Zudem war das Debugging des Zusammenspiels zwischen Java-Backend und Camunda-Engine aufwendig, da Fehler in Service Tasks den Prozess still stoppen können, ohne eine klar sichtbare Fehlermeldung zu erzeugen. Erst durch ausführliches Debug-Logging im Backend wurden viele Probleme sichtbar.

4.2 Reflektion

Das Projekt hat gezeigt, dass Camunda eine leistungsfähige Plattform für die Automatisierung von Geschäftsprozessen ist, die jedoch eine steile Lernkurve mit sich bringt. Die ursprüngliche Erwartung war, dass Camunda als vollständige Engine viele technische Details selbst übernimmt. In der Praxis zeigte sich jedoch, dass Camunda 7 sehr lose gekoppelt und stark vom Backend abhängig ist. Viele Dinge, die man intuitiv der Engine zuschreiben würde, wie Nachrichten zwischen Prozessen weiterleiten oder Fehler sauber propagieren, müssen vollständig selbst implementiert werden. Hinzu kommt, dass Camunda 7.20 bereits als veraltete Version gilt und verschiedene Einschränkungen mitbringt, die in neueren Versionen behoben wurden.

Die Kombination aus BPMN-Prozessmodellierung und externem Java-Backend hat sich trotzdem als sinnvoll erwiesen, da beide Teile unabhängig voneinander entwickelt und getestet werden konnten. Besonders wertvoll war die Erkenntnis, dass ein klar strukturiertes Backend mit definierten Schichten die Entwicklung erheblich erleichtert und spätere Erweiterungen vereinfacht (auch wenn das ohne Camunda ebenso möglich ist). Insgesamt war das Projekt trotz der technischen Schwierigkeiten eine lehrreiche Erfahrung im Umgang mit prozessgetriebenen Systemen.